INNOMATİCS
RESEARCH LABS

Deliverable 2: Low-Level Design (HLD)

Name : Emmadi Srikar
Intern Id : IN226090702
Mail: srikaremmadi5@gmail.com

INNOMATİCS
RESEARCH LABS

RAG-Based Customer Support Assistant With LangGraph & HITL

---

1. Module-Level Design

1. Document Processing Module (rag_pipeline.py)

Responsible for:

- Loading PDF using PyPDFLoader
- Extracting text content
- Preparing data for chunking

---

2. Chunking Module

- Uses RecursiveCharacterTextSplitter
- Splits document into smaller chunks
- Ensures better semantic retrieval and context handling

---

3. Embedding Module

- Uses HuggingFaceEmbeddings
- Converts text chunks into vector representations
- Enables semantic similarity search

---

4. Vector Storage Module

- Uses ChromaDB
- Stores embeddings locally
- Supports efficient similarity-based retrieval

---

5. Retrieval Module

- Retrieves top-k relevant chunks using similarity search

- Applies threshold filtering to remove low-relevance results
- Returns documents along with relevance scores

---

6. Query Processing Module

- Accepts user query
- Combines query with retrieved context
- Sends input to LLM for answer generation

---

7. Graph Execution Module (graph_workflow.py)

- Implements workflow using LangGraph
- Controls execution flow using nodes and edges
- Handles state transitions

---

8. HITL Module (hitl.py)

- Handles escalation when confidence is low
- Returns fallback response simulating human support

---

2. Data Structures

Document Representation

Each document chunk is represented as:

- page_content: text content
- metadata: source, page number, etc.

Chunk Format

- Text-based chunks
- Fixed-size segments created from original document

Embedding Structure

- Numerical vector representation of text
- Stored in ChromaDB for similarity search

Query-Response Schema

```
{
    "query": str,
    "docs": List[Document],
    "answer": str,
    "confidence": float,
    "escalation_flag": bool,
    "escalation_reason": str
}
```

State Object (LangGraph)

```
state = {
    "query": str,
    "docs": list,
    "answer": str,
    "confidence": float,
    "escalation_flag": bool,
    "escalation_reason": str
}
```

---

3. Workflow Design (LangGraph)

Nodes

1. Processing Node ○ Retrieves relevant documents
   - Generates answer using LLM
2. Decision Node ○ Evaluates confidence
   - Determines whether to escalate
3. Output Node ○ Returns final answer to user
4. Escalation Node ○ Handles human fallback response

Edges

- Processing Node → Decision Node
- Decision Node → Output Node (if confident)
- Decision Node → Escalation Node (if uncertain)

State Flow

- State is passed between nodes
- Each node updates specific fields: ○ Processing → answer + confidence ○ Decision → escalation flag
  - Output/Escalation → final response

---

4. Conditional Routing Logic

Routing decision is based on:

1. Confidence Threshold

- If confidence < threshold → escalate

2. Response Validation

- If answer contains "I don't know" → escalate

Logic:

```
is_low_confidence = confidence < CONFIDENCE_THRESHOLD
is_unknown = "i don't know" in answer.lower()

if is_low_confidence or is_unknown:
    escalation_flag = True
```

---

5. HITL Design

When escalation is triggered:

- Low confidence score
- No relevant documents found
- Insufficient answer

After escalation:

- Query is forwarded to HITL module
- System returns predefined human response **Integration:**
- Escalation node replaces LLM response
- Output node returns final response

---

6. API / Interface Design

Input

User Query (string)

Output

Answer (string)

Confidence Score (float)

Escalation Flag (boolean)

Interaction Flow

1. User enters query 2.

System processes query

3. Returns:
    - Answer (if confident)
    - Escalation message (if uncertain)

---

7. Error Handling

1. No Relevant Documents Found

- Trigger escalation
- Provide fallback response

---

2. LLM Failure

- Catch exception
- Return safe fallback message

---

3. Missing Data / API Issues

- Validate inputs
- Log errors using logger

---

8. Summary

The system is designed with modular components that handle document processing, retrieval, and response generation. The use of LangGraph enables structured workflow execution, while conditional routing ensures reliable handling of uncertain queries through HITL escalation.